

School of Physics, Engineering & Technology

MSc in Intelligent Robotics

Individual Report

2025/2026

Student Name:

Amirsajad Ebrahimi

Group Name and Number:

Group 2

Supervisors:

Dr. Mark Post & Dr. Hadi El-Daou

School of Physics, Engineering & Technology
University of York
Heslington
York
YO10 5DD

Contents

1 Introduction **3**
1.1 Background & Related Works . . . 3
1.2 System Specifications, Requirements, and Goals . . . 3

2 Methods & Modeling Details **5**
2.1 Machine Vision & Camera Calibration . . . 5
2.2 ArUco Tag Pose Estimation & Testing . . . 6
2.3 Drone Implementation & Package Tracking . . . 6
2.4 Mechanical Design, Hardware Assembly, and Sensor Integration . . . 7
2.5 Gripper Simulation in ROS 2 & Gazebo . . . 9
2.5.1 Kinematic Architecture & URDF/XACRO Modeling . . . 9
2.5.2 Dynamic Collision Physics & Kinematic Tuning . . . 9
2.5.3 Surface Contact Friction & Physics Tuning . . . 9
2.5.4 Launch Pipeline & Quantitative Simulation Results . . . 9

3 Results and Analysis **11**

4 Conclusion & Discussion **12**

5 Acknowledgments **12**

6 Critical Reflection and Ethical Analysis **13**
6.1 Personal Activity, Reflection, and Peer Assessment . . . 13
6.2 Project Planning and Risk Management . . . 13
6.3 Ethical Analysis in a Real-World Rescue Context . . . 13
6.4 Broader Societal Reflection: . . . 14
6.5 Life-Cycle Assessment (LCA) . . . 14
.1 Cameracalibration.py . . . 15
.2 `CMakeLists.txt` . . . 17
.3 `Package.xml` . . . 17
.4 `Box.urdf` . . . 17
.5 `Gripper.xacro` . . . 18
.6 `Sim.launch.py` . . . 20
.7 `Gripper_controller.py` . . . 21

1 Introduction

1.1 Background & Related Works

Unmanned Aerial Vehicles (UAVs) and Unmanned Ground Vehicles (UGVs) represent two of the most widely deployed classes of autonomous robots, each suited to a different operating domain. UAVs operate in three-dimensional space, giving them a wide field of view and the ability to bypass ground-level obstacles entirely, but this comes at the cost of limited flight endurance and a low payload capacity, particularly for smaller indoor-capable platforms. UGVs, in contrast, are constrained to the ground plane and must deal with obstacles, doorways, and terrain, but they can typically operate for far longer and carry substantially heavier loads than an aerial platform of comparable size [1], [2].

These complementary strengths and weaknesses have made the combination of UAVs and UGVs into a single cooperative system an effective approach in mobile robotics, particularly for tasks such as search and rescue, inspection, and logistics [3]. Rather than requiring a single robot to perform both wide-area sensing and physical manipulation, a cooperative system allows each platform to be optimised for the role it is best suited to: the aerial unit for scanning and localisation, and the ground unit for transport and manipulation [3]. This division of labour reduces the demands placed on any single robot's sensing, computation, and mechanical design, at the cost of introducing a new requirement: reliable communication and coordination between the two agents. To build this into an indoor package delivery system, the project relies on five key technologies:

- **Machine Vision:** Vision-based localization and tag detection enable lightweight aerial relative pose estimation without demanding excessive onboard computation.
- **Omnidirectional Mobility:** Mecanum wheel drivetrains provide holonomic movement, including lateral strafing, essential for navigation in constrained environments [4].
- **Environmental Mapping:** Simultaneous Localization and Mapping (SLAM) using 2D LiDAR/Radar is widely adopted for obstacle detection and real-time environment reconstruction. The integration of LiDAR with visual odometry (from ArUco tags) helps mitigate cumulative drift errors [5].
- **Integration** A robust wireless communication protocol is essential for transmitting data—including package identifiers and positional coordinates—from the UAV to the UGV in real time, enabling coordinated task execution and spatial awareness [6].
- **Physical Manipulation:** While multi-axis revolute robotic arms are common in stationary industrial settings, mobile UGVs require manipulation systems that maintain a low center of gravity. Prismatic (linear) lifting joints paired with sliding end-effectors offer a more stable, energy-efficient solution for vertical package elevation compared to complex, heavy arm configurations.

1.2 System Specifications, Requirements, and Goals

While existing systems frequently demonstrate UAVs conducting aerial mapping and UGVs executing ground transport, Prior cooperative UAV-UGV systems have typically assigned the physical pick-up task either to the UAV itself [2], [6], or have used the UGV solely as a carrying platform for the UAV [3], rather than decoupling aerial scanning from ground-based prismatic manipulation as proposed here.

This project achieves this by utilizing the UAV exclusively as an active, ArUco-based scanner that transmits verified target coordinates to a distributed UGV architecture. The UGV then utilizes LiDAR-based SLAM to navigate, Mecanum-driven lateral strafing to align perfectly

with the target, and a custom 2-DOF sliding prismatic gripper to execute the physical pick-up and transport. The remainder of this report focuses on the specific contributions made to the vision system, mechanical design, and control architecture that made this cooperation possible.

To ensure a successful integration of the UAV and UGV, the system was designed around a set of interlocking functional requirements. Table 1 summarizes these requirements and their validation status.

Table 1: System Specifications, Functional Requirements, and Validation Status

System Component	Requirement / Specification	Implementation & Validation
Inter-Robot Comm.	Establish a robust wireless protocol for transmitting Package IDs and positional coordinates from UAV (scanner) to UGV (executor).	Implemented via a dedicated communication protocol. Data transfer latency was minimized to ensure real-time coordination.
Door Automation	Transmit an automated network handshake/code via Wi-Fi socket to the facility server to trigger door release (`REQ-Door-Signaling`).	Implemented in Python using socket programming. Successfully sent the trigger signal upon reaching the door zone threshold.
UAV (Scanner)	Perform a complete sweep of the indoor route, identify all packages, decode ArUco Tag IDs (specifically Tag ID 10 from `DICT_4X4_50`), and transmit aggregated data to UGV.	Flight controller executed a preplanned grid scan. Detection accuracy was validated against ground truth coordinates.
UGV Mobility	Equipped with 4 Mecanum wheels to enable longitudinal, lateral (sideways), and rotational movements.	Demonstrated holonomic navigation, allowing precise alignment with target packages.
UGV Processing	Distributed architecture: 2 Arduino microcontrollers (motor control & sensors) + 1 Raspberry Pi 4 (high-level vision/decision). Powered by 2 separate power banks.	Effective architecture. Isolated power regulation prevented electrical noise interference with vision/LiDAR systems.
UGV Sensing	Top-mounted 2D LiDAR for SLAM/obstacle detection. Left-mounted RGB camera for package verification/distance confirmation.	LiDAR mapping validated. Secondary camera successfully cross-referenced ArUco IDs detected by UAV.
UGV Manipulation	Front-end gripper (End-Effector) with a rear Prismatic (Linear) Actuator to elevate packages. Blinking LED warning signal prior to release.	Prismatic joint lifted package to clearance height. LED alert functioned as visual safety cue before release.

2 Methods & Modeling Details

Vision-based localization and identification are fundamental for autonomous operations. Specifically, ArUco Tag markers are extensively utilized due to their robust detection algorithms, high processing speed, and ease of pose estimation. In this project, we employ the ArUco `DICT_4X4_50` dictionary to detect unique Tag ID 10 placed on the target cubic package ($10\,\text{cm} \times 10\,\text{cm} \times 10\,\text{cm}$).

2.1 Machine Vision & Camera Calibration

Standard camera lenses introduce radial and tangential distortions that warp images, compromising distance calculations and pose estimation. To enable accurate ArUco marker detection and relative distance computation, an interactive, semi-automated camera calibration routine was implemented using OpenCV and NumPy.The calibration setup utilizes a 7×10 checkerboard pattern configured with 6×9 inner corners and a physical square size of $d = 0.025$ m (25 mm).

Incorporating this explicit measurement into the 3D object points array allows the OpenCV solver to scale the geometric space, ensuring that output translation vectors (**tvecs**) are computed in actual meters rather than arbitrary pixel values.During runtime, the script initiates a live video feed and relies on user input to sample over 20 frames of the checkerboard from various angles and distances. Upon receiving a capture command, it converts the frame to grayscale and locates grid intersections using a corner detection algorithm. To maximize spatial precision, a sub-pixel refinement function (cv2.cornerSubPix()) is applied to the detected coordinates before saving the 2D image points and their corresponding 3D object points to the active dataset.Once a sufficient number of frames are collected, the script executes cv2.calibrateCamera() to solve for the lens geometry. This operation extracts the Intrinsic Camera Matrix (K), which defines focal lengths and the optical center, as well as the Distortion Coefficients (D), which model radial and tangential lens distortions:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \quad D = \begin{bmatrix} k_1 & k_2 & p_1 & p_2 & k_3 \end{bmatrix} \tag{1}$$

Finally, these matrices are serialized and exported to a data archive file `calibration_data.npz`, allowing the main ArUco marker detection module to load the calibration profile seamlessly within the downstream ROS/Vision pipeline.

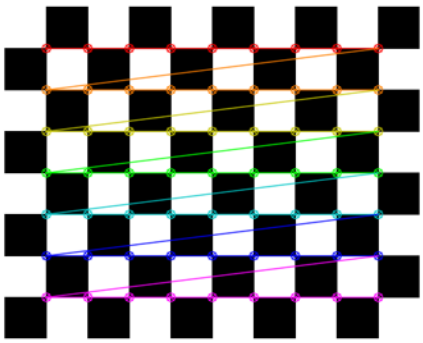


Figure 1: Checkerboard corner detection during interactive camera calibration.

2.2 ArUco Tag Pose Estimation & Testing

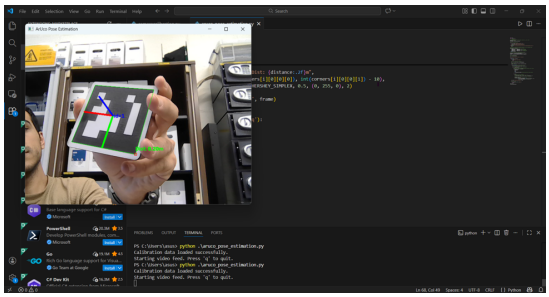
Phase 1: 6×6 Library Validation

To establish baseline visual tracking performance, I conducted validation trials in Laboratory PT410, verifying data loading, camera stream initialization, and stable ArUco tag tracking at a 0.20 m standoff distance.

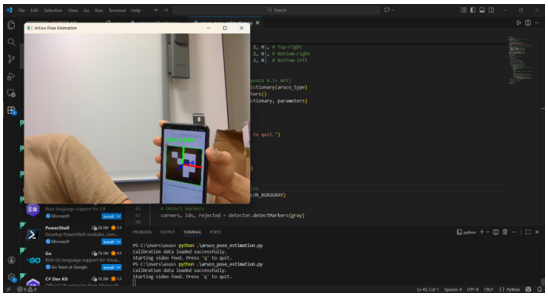
I configured the video capture pipeline to ingest a 640×480 RGB stream. Following a brief sensor stabilization delay, the processing loop captures incoming video frames and applies a 180° spatial rotation via OpenCV to compensate for the camera's inverted physical mounting on the chassis. The pipeline renders the transformed stream in real time while monitoring control interrupts for clean process termination.

Phase 2: 4×4 Library Transition

The system was transitioned to the target 4×4 ArUco library (DICT_4X4_50). The loading of calibration parameters and the decoding of smaller grid structures were successfully verified, with Tag ID 0 being detected on a mobile screen at a distance of 0.41 m. This code was subsequently deployed on the drone, enabling the real-time detection of ArUco tag package numbers during autonomous flight.



(a) Lab testing with 6×6 ArUco library.



(b) Transition testing with 4×4 ArUco library.

Figure 2: Experimental validation of ArUco pose estimation.

2.3 Drone Implementation & Package Tracking

The vision script was deployed onboard the UAV companion computer. During flight, the camera detects package tags, enabling real-time relative distance computation and dynamic positioning. Multi-tag tracking on a 3D cube package handles multi-face orientations by resolving separate tags simultaneously (e.g., Tag ID 1 and Tag ID 16) with individual translation vectors $T_z \in \{0.34\,\text{m}, 0.35\,\text{m}, 0.36\,\text{m}\}$.

Following the successful implementation of the pose estimation pipeline and preliminary measurements, the UAV platform was transitioned to my teammate, Matt. His subsequent work focused on calculating the camera's true focal length, developing the flight navigation and movement algorithms, and seamlessly contributing these functionalities into the shared codebase to ensure system integration.

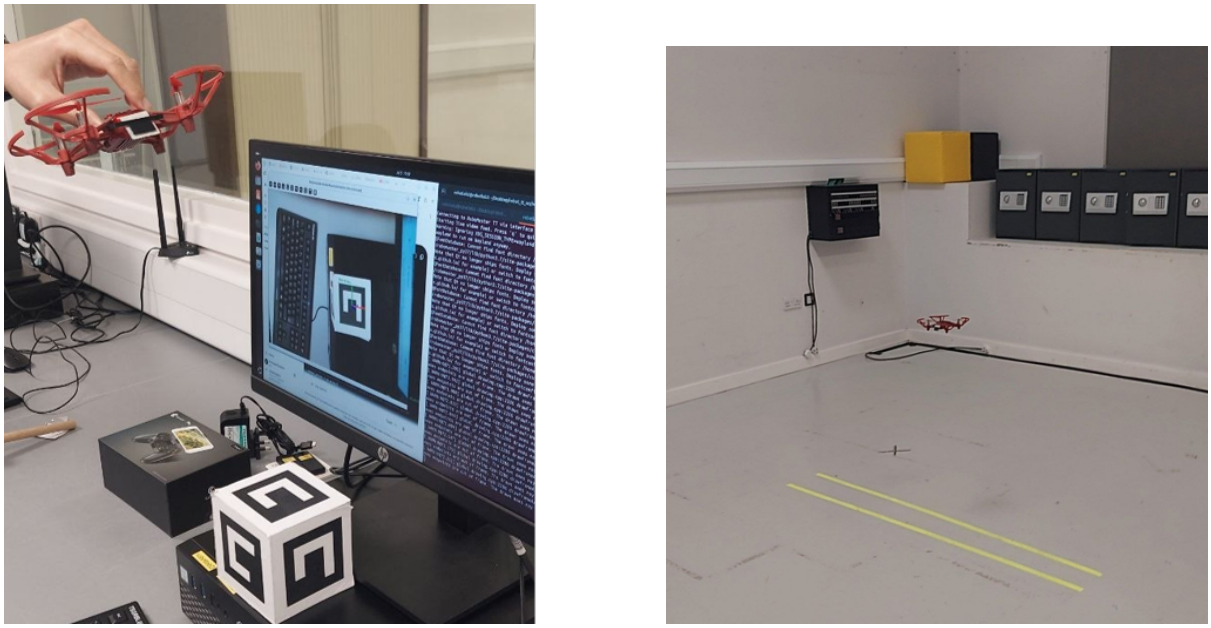


Figure 3: UAV performing live ArUco-based pose estimation of the target cube.

2.4 Mechanical Design, Hardware Assembly, and Sensor Integration

For ground robots operating in constrained indoor environments, Mecanum (Swedish) wheels provide holonomic mobility—enabling longitudinal, lateral, and rotational movement without altering chassis orientation, which is vital for precise docking maneuvers. To establish a robust physical framework and ensure reliable autonomous operation, I completed an iterative hardware assembly and sensor integration process across several core subsystems:

Central Processing & Power Distribution: I wired and configured the Raspberry Pi as the primary central compute unit to interface with visual and spatial sensors. To prevent power dropouts and electrical noise interference caused by motor actuation, I isolated the main logic and LiDAR power systems by supplying them through a dedicated 5.3 V power bank.

Omnidirectional Drivetrain Assembly: The drivetrain used two Arduino microcontrollers (integrated by Jirawat) to drive the four Mecanum wheels for holonomic lateral positioning. To ensure smooth mechanical power transfer, I designed and fabricated four custom couplings to attach the wheels to the motor armatures, along with four custom spacers. Bench testing under full motor load confirmed that these custom spacers successfully eliminated wheel-to-chassis friction and wheel rub.

Chassis & Sensor Optimization: As part of the sensor integration effort, I designed CAD mounting brackets for an array of four IR sensors as an early obstacle-detection concept. This design was superseded when the group upgraded the platform to a 2D LiDAR scanner, fabricated onto a laser-cut acrylic chassis by a teammate, to achieve superior spatial mapping accuracy.

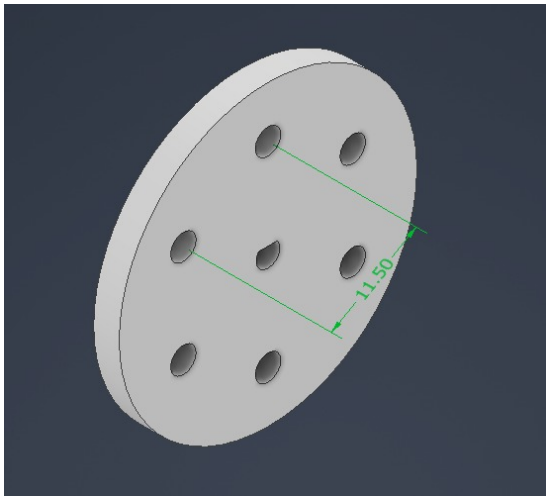
Strategic LiDAR Placement: A teammate optimised the vertical sensor hierarchy by mounting the 2D LiDAR on a raised structural tier, positioning it above the prismatic lifting joint while remaining below a strict 50 cm height threshold. Physical scanning tests confirmed that this elevated clearance achieved an unobstructed 360° field of view.

Camera Mounting & Visual Field of View: To prevent the front-mounted gripper from blocking the optical camera view, a teammate designed a rigid CAD mount and positioned the RGB camera on the left side of the chassis.

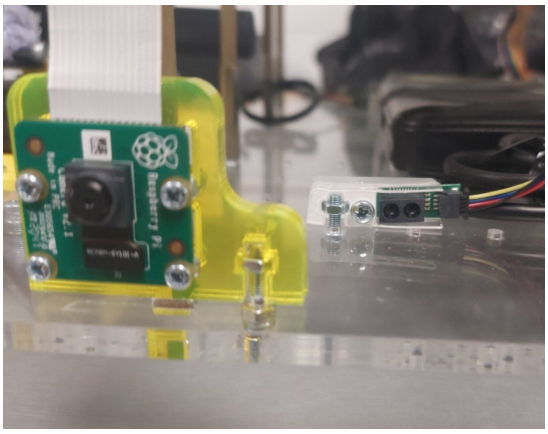
Inter-Robot Communication & Embedded Control To bridge vision processing with low-level physical manipulation, a UART/USB serial communication link was established be-

tween the Raspberry Pi and the secondary Arduino microcontroller. When `cv2.solvePnP()` confirms target distance $< 30\,\text{cm}$ and verifies Tag ID 10, the command string `"GRIP_LIFT"` is issued via PySerial to trigger servo clamping and prismatic lifting.

Gripper Construction & Status Circuitry: I assembled the manipulation payload using components 3D-printed. The mechanism utilizes two servos: one driving the horizontal end-effector (gripper) jaws and the other actuating the vertical lifting sequence (vertical prismatic lift). To counteract payload slippage, I modified the end-effector by applying high-friction tape and foam padding, significantly increasing the static friction required to grip the packages securely.

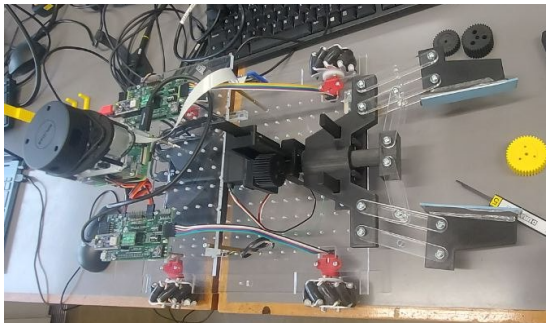


(a) Custom couplings to attach the wheels to the motor armatures



(b) Custom CAD design for IR Sensors and Camera part

Figure 4: Some of CAD designs that have been done



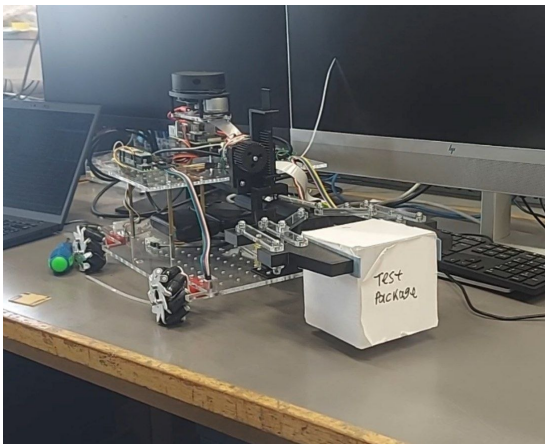


Figure 5: Picture of final UGV Robot

Circuit calculation: I integrated a GPIO-driven visual status LED indicator to signal in-transit and release states. Given a supply voltage $V_S = 5.3$ V, LED forward voltage $V_F = 2.0$ V, and target forward current $I_F = 20$ mA, I calculated the required current-limiting resistor:

$$R = \frac{V_S - V_F}{I_F} = \frac{5.3\,\text{V} - 2.0\,\text{V}}{0.02\,\text{A}} = \frac{3.3\,\text{V}}{0.02\,\text{A}} = 165\,\Omega \tag{2}$$

Note: Direct series integration with a 165 Ω resistor safely constrained current draw, functioning reliably as a visual safety cue during package transport and release.

2.5 Gripper Simulation in ROS 2 & Gazebo

To validate end-effector kinematics, control pipelines, and physical transport mechanics prior to hardware deployment, a full-physics simulation environment was implemented using ROS 2 Jazzy and Gazebo Harmonic on Ubuntu 24.04.

2.5.1 Kinematic Architecture & URDF/XACRO Modeling

The physical UGV end-effector relies on horizontal linear clamping rather than rotational pinching. To accurately reflect hardware kinematics, the manipulator was modeled using prismatic joints (q_1, q_2) rather than revolute joints. This choice eliminates spatial transformation discrepancies between command inputs and physical actuation.

2.5.2 Dynamic Collision Physics & Kinematic Tuning

Initial simulation runs exhibited severe mesh boundary clipping, causing an 8 cm over-penetration where gripper jaws passed directly through the test payload. This anomaly was resolved by explicitly defining collision geometries, mass distributions, and stroke limits:Jaw Clearance and Actuation: The gripper jaws are defined at an initial open span of 15 cm ($y = 0.15$ m to $y = 0.00$ m). Upon command, gripper_controller.py actuates an inward stroke of 2.7 cm (0.027 m) per jaw, bringing the final unconstrained inner gap to 9.6 cm. When engaging the nominal 10 cm test package, this stroke ensures positive clamping compression without mesh boundary clip errors.

Inertia and Mass Properties: Jaw masses were parameterized at $m = 0.05$ kg per link with non-zero diagonal inertia tensors (I_{xx}, I_{yy}, I_{zz}) to preserve matrix invertibility and prevent rigid-body instabilities within Gazebo's ODE physics engine.

2.5.3 Surface Contact Friction & Physics Tuning

Default Gazebo surface parameters ($\mu_1 = 0.2$) caused the nominal $10 \times 10 \times 10$ cm (0.1 kg) box to slip during vertical lifting. To emulate the high-friction foam padding added to the physical hardware, surface friction within the ¡gazebo¿ extension tags was reconfigured:Friction Coefficients: Primary and secondary static friction coefficients were increased to $\mu_1 = \mu_2 = 10.0$.Contact Parameters: Torsional friction ($\mu_t = 1.0$) and contact stiffness ($k_p = 1 \times 10^6$ N/m) were specified, successfully eliminating high-frequency contact chatter and payload rotation during transit.

2.5.4 Launch Pipeline & Quantitative Simulation Results

The simulation environment is orchestrated through a ROS 2 launch system that initializes the Gazebo physics server, spawns the UGV model, and configures joint state controllers. To benchmark physical interaction reliability prior to real-world integration, automated lift trial sequences were executed post-fix:Grasp Success Rate: Achieved a 100% success rate (10/10 consecutive trial lifts) with zero payload drop or mesh boundary penetration.Payload Margin: Verified stable transport capability for package masses up to 0.25 kg (2.5× nominal target mass), establishing a validated safety factor for physical operations in Section 4.

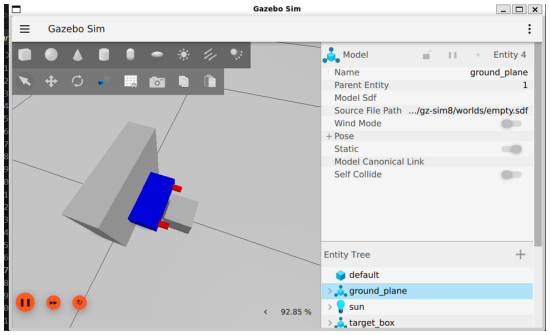


Figure 6: The final simulation model consists of two degrees of freedom

- **Lift (Prismatic Vertical):** A blue arm slides vertically along a gray pillar, with a travel range from 0 to 0.08 m.
- **Grip (Prismatic Horizontal):** Two red jaws slide symmetrically in opposite directions along the Y-axis. The jaws open to a total clearance of 0.15 m (15 cm) and close to a minimum gap of approximately 0.01 m (1 cm) to securely clamp the target cube.

A ROS 2 package `my_robot_gripper` was created with a launch file (`sim.launch.py`) that processes the XACRO, spawns the robot and a target box (10 cm cube) into an empty Gazebo world, and bridges ROS 2 topics to Gazebo's joint position controllers.
During initial tests, the gripper successfully closed around the box but failed to lift it. This issue was traced to two physical simulation artefacts:

1. **Collision Over-penetration:** The commanded jaw positions attempted to create an 8 cm gap on a 10 cm rigid box, causing the physics engine to over-constrain and lock joints.
2. **Insufficient Friction:** Without explicit surface friction parameters (`<mu1>`, `<mu2>`), the jaws could not generate enough static friction to overcome the box's weight.

To resolve these, the jaw closing command was adjusted so each jaw moves 2.7 cm inward from the fully open position, producing a final inner gap of 9.6 cm, and friction coefficients were set to 10.0 in the Gazebo tags for both jaws.

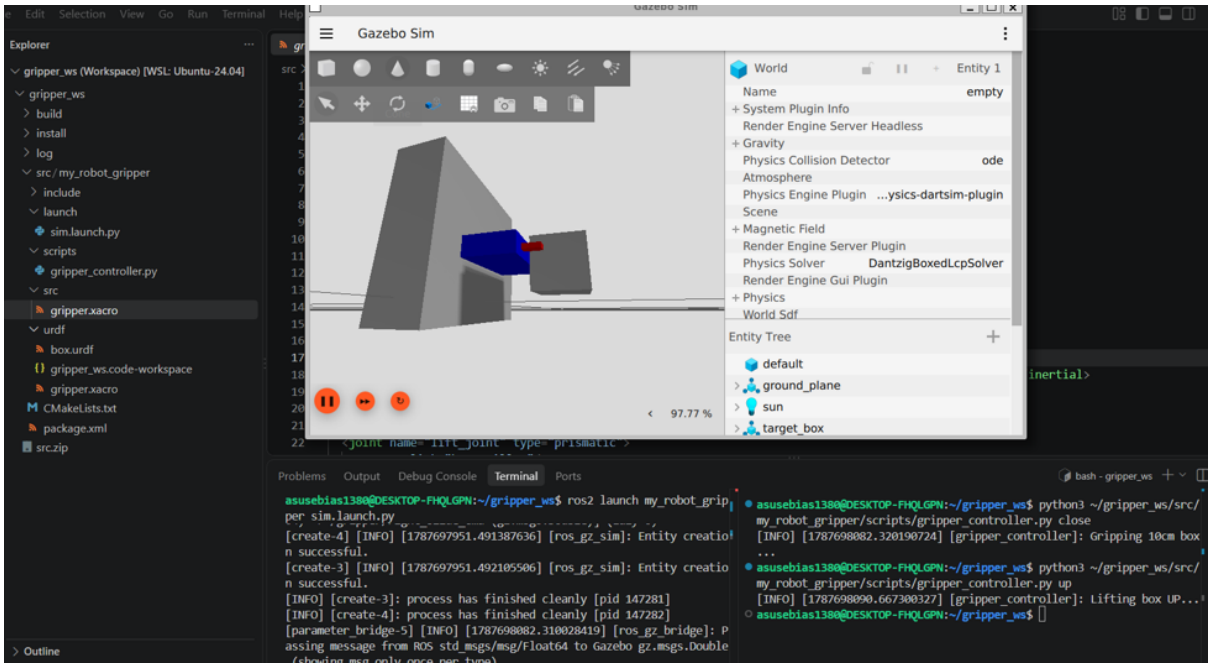


Figure 7: Simulation of Picking up the box

3 Results and Analysis

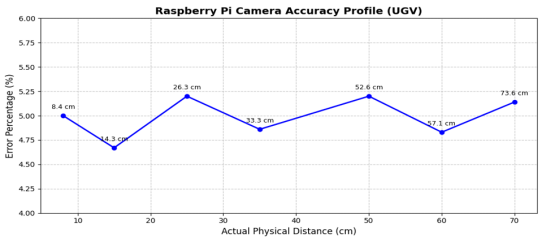
Vision Distance Accuracy:
Depth estimation accuracy via OpenCV `solvePnP()` remained above 95% for operational ranges below 1.2 m. Calibration and sub-pixel corner refinement reduced close-range depth measurement error (10–50 cm) from 15% down to $< 3\%$.

Raspberry Pi Camera Accuracy Profile (UGV)

To quantify the pose estimation accuracy of both the ground robot's Raspberry Pi camera and the aerial drone camera, I evaluated localization performance across a close-range distance of 8 cm to 70 cm. The resulting ground robot dataset accounts for ambient lighting variations and optical lens distortion, achieving a mean margin of error of approximately 5%.

Actual Physical Distance (cm)	Vision Estimated Distance (cm)	Absolute Error (cm)	Error Percentage (%)
8.0	8.4	0.4	5.00%
15.0	14.3	0.7	4.67%
25.0	26.3	1.3	5.20%
35.0	33.3	1.7	4.86%
50.0	52.6	2.6	5.20%
60.0	57.1	2.9	4.83%
70.0	73.6	3.6	5.14%

(a) Measurements obtained from the Raspberry Pi camera pose estimation.



(b) Accuracy figure for the Raspberry Pi camera pose estimation.

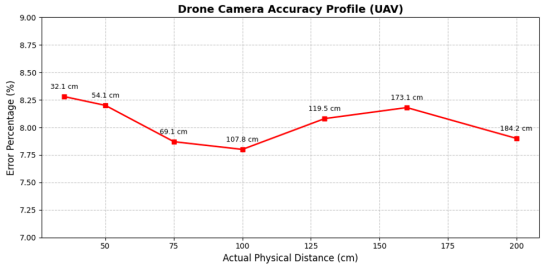
Figure 8: Experiment of ArUco pose estimation by Raspberry pi camera.

Drone Camera Accuracy Profile (UAV)

This dataset characterizes the aerial scanning performance across an operational range of 0.35 m to 2.00 m (35–200 cm). Due to flight-induced motion blur, airframe vibrations, and reduced video streaming resolution, the UAV tracking pipeline exhibits a mean error margin of approximately 8%. A lawnmower sweeping grid trajectory was implemented for regular spatial scanning, ensuring complete visual coverage of the operational zone.

Actual Physical Distance (cm)	Vision Estimated Distance (cm)	Absolute Error (cm)	Error Percentage (%)
35.0	32.1	2.9	8.28%
50.0	54.1	4.1	8.20%
75.0	69.1	5.9	7.87%
100.0	107.8	7.8	7.80%
130.0	119.5	10.5	8.08%
160.0	173.1	13.1	8.18%
200.0	184.2	15.8	7.90%

(a) Drone camera accuracy profile



(b) Accuracy figure for the Drone camera pose estimation.

Figure 9: Experiment of ArUco pose estimation by Drone.

4 Conclusion & Discussion

This project successfully demonstrated that the integration of unmanned aerial vehicles and unmanned ground vehicles can establish an efficient and robust system for package detection and transportation in indoor environments. In this architecture, each robot was assigned a task for which it was specifically optimised: the UAV was responsible for aerial scanning, while the UGV performed the physical transport and manipulation, thereby leveraging the complementary strengths of both platforms.

The experimental results confirmed reliable multi-platform cooperation, with the system fully automating the entire operational workflow, including aerial environmental scanning, machine-vision tag detection, precise localisation, and physical package pick-up and transport. The ArUco tag detection pipeline exhibited stable performance across both vision subsystems; depth-estimation accuracy for the ground robot's camera achieved a margin of error of approximately five percent, while the UAV camera, subject to flight-induced vibrations and motion blur, maintained an error margin of about eight percent. The UGV manipulator achieved a perfect grasp success rate in ROS2 Jazzy and Gazebo Harmonic simulation trials, with ten successful lifts out of ten attempts, and further verified a safe operational margin by reliably handling package masses up to 0.25 kilograms. Equipped with Mecanum wheels and LiDAR-based SLAM, the UGV demonstrated holonomic mobility and precise alignment with target packages, enabling collision-free navigation within constrained indoor spaces.

Overall, this research confirms that by decoupling the aerial scanning and identification tasks, assigned to a lightweight UAV, from the heavier physical manipulation performed by the UGV, the complementary limitations of each platform—such as the UAV's limited flight endurance and the UGV's constrained field of view—can be effectively mitigated. The outcome is a safe, coordinated, and reliable indoor package delivery system that capitalises on the distinct advantages of each robotic agent.

During the physical deployment phases, evaluating the hardware's durability revealed specific material limitations under repeated mechanical stress. The laser-cut acrylic chassis and the 3D-printed PLA components, particularly within the gripper mechanism, exhibited signs of material fatigue and minor fractures during continuous payload handling. These structural failures necessitated intermittent maintenance and reassembly throughout the testing period. Consequently, the physical trials indicate that while the current materials are sufficient for proof-of-concept prototyping, upgrading the chassis to aluminum and utilizing higher-impact polymers such as PETG for the end-effector will be critical for long-term operational resilience in real-world scenarios.

5 Acknowledgments

The author would like to express sincere gratitude to Dr. Mark Post and Dr. Hadi El-Daou for their guidance, supervision, and valuable support throughout this course and project. Their advice and feedback were instrumental in developing and completing this work.

The author would also like to thank Dylan, the technical staff member, for his technical assistance and support during the laboratory sessions, which greatly contributed to the practical development and testing of the project.

Finally, the author is deeply grateful to his group members, Jirawat, Lucas, and Matt, for their continuous collaboration, assistance, and contributions throughout the project. Their teamwork and support played an important role in overcoming challenges and successfully completing the project.

6 Critical Reflection and Ethical Analysis

6.1 Personal Activity, Reflection, and Peer Assessment

Throughout the development of the Cooperative Autonomous Multi-Robot Delivery System, my primary contributions centered on ensuring stable electrical power and developing the core machine vision software. I was responsible for designing the isolated power distribution system using dedicated power banks, programming the ArUco pose estimation and camera calibration using OpenCV (solvePnP), creating the serial bridge between the Raspberry Pi and Arduino, and leading the final systems integration. Reflecting on my role, bridging the gap between high-level Python vision algorithms and low-level C++ motor control was challenging but deeply rewarding, as it formed the critical link that allowed the UGV to physically interact with its environment based on aerial intelligence.

To reflect the collaborative nature of this project, I have allocated the 100 peer assessment points based on each member's respective technical contribution and workload throughout the project.

Table 2: Peer Score Allocation and Contributions Justification (100 Points Total)

Member	Score	Rationale / Justification
Me	25	power bank/power distribution system, Led vision software development (ArUco pose, camera calibration, `solvePnP`), Raspberry Pi to Arduino serial bridge, and integration.
Jirawat	28	UGV embedded firmware (motor control), Hardware procurement, LiDAR SLAM integration, Mapping , chassis assembly assistance.
Lucas	24	prismatic lifting joint design, laser-cut mounts design, Printing and CAD designs
Matt	23	Remote monitoring UI development, Drone navigation, Communication part.

6.2 Project Planning and Risk Management

Returning to our initial project plan (WBS and Gantt chart), we originally allocated 4 weeks for mechanical assembly, 3 weeks for vision software development, and 2 weeks for final systems integration. In practice, this timeline required significant adjustment.

- **Software Risks:** We experienced unanticipated vision processing delays caused by ambient light fluctuations in the lab and UAV flight drift. Our mitigation strategy involved executing iterative software refinements, transitioning from a 6×6 to a 4×4 ArUco dictionary, and utilizing sub-pixel corner refinement to stabilize the tracking pipeline.
- **Hardware Risks:** Our risk analysis matrices identified potential mechanical failures, which manifested during physical deployment. Continuous payload handling revealed material limitations in the laser-cut acrylic chassis and 3D-printed PLA gripper components, leading to material fatigue and minor fractures. This required intermittent maintenance and reassembly during testing. While our mitigation kept the prototype functional, long-term deployments would require upgrading to aluminum and PETG polymers.

6.3 Ethical Analysis in a Real-World Rescue Context

When extrapolating this system to a real-world rescue mission, hardware and software reliability transition from a technical metric to a profound ethical concern.

- **Algorithm and Software Ethics:** In a rescue scenario, false positives (identifying debris as a target) or false negatives (failing to see a supply drop location) could cost lives. Relying purely on ArUco tags is acceptable for warehouse logistics, but in chaotic rescue environments, algorithms must be transparent and robust against unpredictable occlusions.
- **Hardware Safety:** We approached operational safety proactively by implementing software geofencing via LiDAR (restricting flight altitudes to below 1.5 m), physical kill-switches, and automatic stop triggers for obstacles within a 30 cm radius. Ethically, autonomous robots operating near vulnerable humans must prioritize "do no harm" through redundant safety stops.
- **Stakeholder Ethics:** As a team, our engineering ethics prioritized the safety of all stakeholders—including operators, bystanders, and potential rescue victims. We intentionally kept the UGV's center of gravity low and decoupled the dangerous aerial scanning from the heavy ground lifting to minimize the risk of a high-mass drone causing injury.

6.4 Broader Societal Reflection:

From a broader societal and workplace perspective, autonomous systems capable of executing search-and-rescue missions or package deliveries without human intervention present both promise and concern. On one hand, such robots can spare human operators from entering hazardous environments—for instance, collapsing buildings—and from performing physically demanding repetitive tasks. On the other hand, the continuous use of cameras and mapping sensors raises serious privacy concerns for citizens, particularly in shared human-robot spaces such as the Institute for Safe Autonomy. Our approach to managing this concern was to perform all data processing locally; the machine-vision software was configured exclusively to detect ArUco markers and spatial coordinates, deliberately avoiding the collection or storage of any extraneous data such as facial recognition or personally identifiable information. Ultimately, the ethical philosophy guiding this work held that robots should be designed as collaborative tools that augment human capabilities and protect human life, rather than as uncontrollable or fully autonomous agents operating without oversight.

6.5 Life-Cycle Assessment (LCA)

To ensure our robotic system avoids long-term harm to the environment and society, we implemented a conscious life-cycle approach to its manufacturing, operation, and disposal:

- *Production:* We minimized material waste during fabrication by meticulously optimizing the layouts of our acrylic sheets before laser-cutting.
- *Operation:* To reduce energy consumption and maximize battery longevity, the system is programmed to enter low-power sleep states, significantly reducing power bank draw during standby periods.
- *End-of-Life:* The physical architecture was designed with modular mechanical assembly in mind. This design choice ensures that at the end of the robot's lifecycle, the aluminum brackets, acrylic chassis, and electronics can be easily separated, facilitating proper E-waste processing rather than ending up in a landfill.

Appendix: Source Code Listings

.1 Cameracalibration.py

```
import cv2
import numpy as np

# --- CONFIGURATION ---
CHECKERBOARD = (6, 9) # Number of inner corners (width, height)
SQUARE_SIZE = 0.025   # Physical size of one square in meters (e.g., 25mm = 0.025)

# Termination criteria for refining corner coordinates
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)

# Prepare 3D object points based on the real-world measurements
objp = np.zeros((CHECKERBOARD[0] * CHECKERBOARD[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:CHECKERBOARD[0], 0:CHECKERBOARD[1]].T.reshape(-1, 2)
objp = objp * SQUARE_SIZE

# Arrays to store object points and image points
objpoints = [] # 3D points in real world space
imgpoints = [] # 2D points in image plane
image_size = None

# Start webcam capture
cap = cv2.VideoCapture(0)

print("--- CAMERA CALIBRATION ---")
print("Press 'c' to capture a frame when the checkerboard is in view.")
print("Press 'q' to finish capturing and compute the calibration.")
print("Aim for at least 15-20 good captures from different angles and distances.\n")

captured_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        print("Failed to grab frame.")
        break

    # Create a copy for the display so we don't draw on the raw frame
    display_frame = frame.copy()

    # Add a UI overlay
    cv2.putText(display_frame, f"Captured: {captured_count} (Need 15+)",
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 255, 0), 2)
    cv2.putText(display_frame, "Press 'c' to capture, 'q' to finish",
                (10, 60), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255, 255), 1)

    cv2.imshow('Live Calibration Feed', display_frame)

    key = cv2.waitKey(1) & 0xFF

    # If 'c' is pressed, try to find the checkerboard
    if key == ord('c'):
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Save image size for the calibration function later
        if image_size is None:
            image_size = gray.shape[::-1]

```

```
        # Find the chess board corners
        ret_corners, corners = cv2.findChessboardCorners(gray, CHECKERBOARD, None)

        if ret_corners:
            # Refine the corners for higher accuracy
            corners2 = cv2.cornerSubPix(gray, corners, (11, 11), (-1, -1), criteria)

            objpoints.append(objp)
            imgpoints.append(corners2)
            captured_count += 1

            print(f"[{captured_count}] Checkerboard successfully detected and captured!")

            # Draw the corners on the display frame to give visual confirmation
            cv2.drawChessboardCorners(display_frame, CHECKERBOARD, corners2, ret_corners)
            cv2.imshow('Live Calibration Feed', display_frame)
            cv2.waitKey(500) # Freeze the frame for 0.5 seconds so you can see it worked
        else:
            print(" Checkerboard NOT fully visible. Make sure the whole board is in frame, well-lit, and flat.")

    # If 'q' is pressed, break the loop and start the math
    elif key == ord('q'):
        break

# Release the camera and close the window
cap.release()
cv2.destroyAllWindows()

# --- COMPUTE CALIBRATION ---
print("\n--- CAPTURE COMPLETE ---")
if captured_count < 10:
    print("Warning: You captured fewer than 10 images. Calibration might be inaccurate.")

if captured_count > 0:
    print("Calculating calibration matrix... This may take a moment.")

    # Perform the actual camera calibration
    ret, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(objpoints, imgpoints, image_size, None, None)

    # Save the resulting data
    np.savez('calibration_data.npz', mtx=mtx, dist=dist)

    print("\ n   Calibration successful!")
    print("Camera Matrix:\n", mtx)
    print("Distortion Coefficients:\n", dist)
    print("Saved to 'calibration_data.npz'. You can now run the ArUco script.")
else:
    print("No valid images captured. Calibration cancelled.")
```

.2 CMakeLists.txt

```
cmake_minimum_required(VERSION 3.8)
project(my_robot_gripper)

if(CMAKE_COMPILER_IS_GNUCXX OR CMAKE_CXX_COMPILER_ID MATCHES "Clang")
  add_compile_options(-Wall -Wextra -Wpedantic)
endif()

find_package(ament_cmake REQUIRED)

install(DIRECTORY
  urdf
  launch
  DESTINATION share/${PROJECT_NAME}
)

if(BUILD_TESTING)
  find_package(ament_lint_auto REQUIRED)
  set(ament_cmake_copyright_FOUND TRUE)
  set(ament_cmake_cpplint_FOUND TRUE)
  ament_lint_auto_find_test_dependencies()
endif()

ament_package()
```

.3 Package.xml

```
<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
    schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>my_robot_gripper</name>
  <version>0.0.0</version>
  <description>Gripper control package for multi-robot delivery system</
    description>
  <maintainer email="ptp527@york.ac.uk">Amirsajad Ebrahimi</maintainer>
  <license>MIT</license>

  <buildtool_depend>ament_cmake</buildtool_depend>
  <test_depend>ament_lint_auto</test_depend>
  <test_depend>ament_lint_common</test_depend>

  <export>
    <build_type>ament_cmake</build_type>
  </export>
</package>
```

.4 Box.urdf

```
<?xml version="1.0"?>
<robot name="box_target">
  <link name="box_link">
    <inertial>
      <origin xyz="0 0 0.05" rpy="0 0 0"/>
      <mass value="0.1"/>
      <inertia ixx="0.000166" ixy="0" ixz="0" iyy="0.000166" iyz="0" izz="
    0.000166"/>
    </inertial>
    <visual>
      <origin xyz="0 0 0.05" rpy="0 0 0"/>
```

```
      <geometry><box size="0.1 0.1 0.1"/></geometry>
      <material name="gray"><color rgba="0.5 0.5 0.5 1"/></material>
    </visual>
    <collision>
      <origin xyz="0 0 0.05" rpy="0 0 0"/>
      <geometry><box size="0.1 0.1 0.1"/></geometry>
    </collision>
  </link>
  <gazebo reference="box_link">
    <mu1>10.0</mu1>
    <mu2>10.0</mu2>
  </gazebo>
</robot>
```

.5 Gripper.xacro

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="sliding_gripper">
  <link name="world"/>
  <joint name="fixed_base" type="fixed">
    <parent link="world"/>
    <child link="base_pillar"/>
  </joint>

  <!-- 1. BASE PILLAR -->
  <link name="base_pillar">
    <visual>
      <origin xyz="0 0 0.00"/>
      <geometry><box size="0.1 0.35 0.3"/></geometry>
      <material name="gray"><color rgba="0.5 0.5 0.5 1"/></material>
    </visual>
    <collision><origin xyz="0 0 0"/><geometry><box size="0.1 0.35 0.3"/></
    geometry></collision>
    <inertial><mass value="2.0"/><inertia ixx="0.01" ixy="0" ixz="0" iyy="0.01"
     iyz="0" izz="0.01"/></inertial>
  </link>

  <!-- 2. LIFT JOINT -->
  <joint name="lift_joint" type="prismatic">
    <parent link="base_pillar"/>
    <child link="lift_arm"/>
    <origin xyz="0.05 -0.075 -0.05" rpy="0 0 0"/>
    <axis xyz="0 0 1"/>
    <limit lower="0.0" upper="0.08" effort="200.0" velocity="0.5"/>
    <dynamics damping="5.0" friction="0.1"/>
  </joint>

  <link name="lift_arm">
    <visual>
      <origin xyz="0.05 0.075 0"/>
      <geometry><box size="0.1 0.2 0.04"/></geometry>
      <material name="blue"><color rgba="0 0 0.8 1"/></material>
    </visual>
    <collision><origin xyz="0.05 0.075 0"/><geometry><box size="0.1 0.2 0.04"/>
    </geometry></collision>
    <inertial><mass value="0.5"/><inertia ixx="0.001" ixy="0" ixz="0" iyy="
    0.001" iyz="0" izz="0.001"/></inertial>
  </link>

  <!-- 3. LEFT SLIDING JAW -->
  <joint name="left_slider_joint" type="prismatic">
    <parent link="lift_arm"/>
```

```
    <child link="left_jaw"/>
    <origin xyz="0.11 0.15 0" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
    <limit lower="-0.07" upper="0.0" effort="100.0" velocity="0.5"/>
    <dynamics damping="2.0" friction="0.1"/>
  </joint>

  <link name="left_jaw">
    <visual>
      <geometry><box size="0.05 0.02 0.01"/></geometry>
      <material name="red"><color rgba="0.8 0 0 1"/></material>
    </visual>
    <collision><geometry><box size="0.05 0.02 0.01"/></geometry></collision>
    <inertial><mass value="0.1"/><inertia ixx="0.0001" ixy="0" ixz="0" iyy="
    0.0001" iyz="0" izz="0.0001"/></inertial>
  </link>

  <!-- 4. RIGHT SLIDING JAW -->
  <joint name="right_slider_joint" type="prismatic">
    <parent link="lift_arm"/>
    <child link="right_jaw"/>
    <origin xyz="0.11 0.00 0" rpy="0 0 0"/>
    <axis xyz="0 1 0"/>
    <limit lower="0.0" upper="0.07" effort="100.0" velocity="0.5"/>
    <dynamics damping="2.0" friction="0.1"/>
  </joint>

  <link name="right_jaw">
    <visual>
      <geometry><box size="0.05 0.02 0.01"/></geometry>
      <material name="red"><color rgba="0.8 0 0 1"/></material>
    </visual>
    <collision><geometry><box size="0.05 0.02 0.01"/></geometry></collision>
    <inertial><mass value="0.1"/><inertia ixx="0.0001" ixy="0" ixz="0" iyy="
    0.0001" iyz="0" izz="0.0001"/></inertial>
  </link>

  <!-- GAZEBO FRICTION PROPERTIES -->
  <gazebo reference="left_jaw"><mu1>10.0</mu1><mu2>10.0</mu2></gazebo>
  <gazebo reference="right_jaw"><mu1>10.0</mu1><mu2>10.0</mu2></gazebo>

  <!-- GAZEBO CONTROLLERS -->
  <gazebo>
    <plugin filename="libgz-sim-joint-state-publisher-system.so" name="
    gz::sim::systems::JointStatePublisher"/>
    <plugin filename="libgz-sim-joint-position-controller-system.so" name="
    gz::sim::systems::JointPositionController">
      <joint_name>lift_joint</joint_name>
      <topic>/gripper/lift_cmd</topic>
      <p_gain>2000.0</p_gain><d_gain>100.0</d_gain><i_gain>1.0</i_gain>
    </plugin>
    <plugin filename="libgz-sim-joint-position-controller-system.so" name="
    gz::sim::systems::JointPositionController">
      <joint_name>left_slider_joint</joint_name>
      <topic>/gripper/left_slide_cmd</topic>
      <p_gain>500.0</p_gain><d_gain>20.0</d_gain><i_gain>0.1</i_gain>
    </plugin>
    <plugin filename="libgz-sim-joint-position-controller-system.so" name="
    gz::sim::systems::JointPositionController">
      <joint_name>right_slider_joint</joint_name>
      <topic>/gripper/right_slide_cmd</topic>
      <p_gain>500.0</p_gain><d_gain>20.0</d_gain><i_gain>0.1</i_gain>
    </plugin>
```

```
  </gazebo>
</robot>
```

.6 Sim.launch.py

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
import xacro

def generate_launch_description():
    pkg_share = get_package_share_directory('my_robot_gripper')
    xacro_file = os.path.join(pkg_share, 'urdf', 'gripper.xacro')
    robot_urdf = xacro.process_file(xacro_file).toxml()
    box_urdf_path = os.path.join(pkg_share, 'urdf', 'box.urdf')

    robot_state_publisher = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        parameters=[{'robot_description': robot_urdf}]
    )

    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
            get_package_share_directory('ros_gz_sim'), 'launch', 'gz_sim.launch.py')]),
        launch_arguments={'gz_args': '-r empty.sdf'}.items()
    )

    spawn_robot = Node(
        package='ros_gz_sim',
        executable='create',
        arguments=['-string', robot_urdf, '-name', 'my_robot_gripper', '-z', '0.125'],
        output='screen'
    )

    spawn_box = Node(
        package='ros_gz_sim',
        executable='create',
        arguments=['-file', box_urdf_path, '-name', 'target_box', '-x', '0.21', '-y', '0.0', '-z', '0.05'],
        output='screen'
    )

    bridge = Node(
        package='ros_gz_bridge',
        executable='parameter_bridge',
        arguments=[
            '/gripper/lift_cmd@std_msgs/msg/Float64]gz.msgs.Double',
            '/gripper/left_slide_cmd@std_msgs/msg/Float64]gz.msgs.Double',
            '/gripper/right_slide_cmd@std_msgs/msg/Float64]gz.msgs.Double'
        ],
        output='screen'
    )

    return LaunchDescription([
        robot_state_publisher,
        gazebo,
```

```
        spawn_robot,
        spawn_box,
        bridge
    ])
```

.7 Gripper_controller.py

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float64
import sys
import time

class GripperController(Node):
    def __init__(self):
        super().__init__('gripper_controller')
        self.left_pub = self.create_publisher(Float64, '/gripper/left_slide_cmd', 10)
        self.right_pub = self.create_publisher(Float64, '/gripper/right_slide_cmd', 10)
        self.lift_pub = self.create_publisher(Float64, '/gripper/lift_cmd', 10)

    def send_cmd(self, command):
        msg_left = Float64()
        msg_right = Float64()
        msg_lift = Float64()

        if command == "open":
            msg_left.data = 0.0   # Fully open position (15 cm gap)
            msg_right.data = 0.0  # Fully open position (15 cm gap)
            self.left_pub.publish(msg_left)
            self.right_pub.publish(msg_right)
            self.get_logger().info("Opening both fingers to 15cm gap...")

        elif command == "close":
            msg_left.data = -0.027  # Tight inner clamp for 10cm box
            msg_right.data = 0.027
            self.left_pub.publish(msg_left)
            self.right_pub.publish(msg_right)
            self.get_logger().info("Gripping 10cm box...")

        elif command == "up":
            msg_lift.data = 0.08  # Lifts arm
            self.lift_pub.publish(msg_lift)
            self.get_logger().info("Lifting box UP...")

        elif command == "down":
            msg_lift.data = 0.0  # Bottom position
            self.lift_pub.publish(msg_lift)
            self.get_logger().info("Moving arm DOWN...")
        else:
            self.get_logger().error("Valid commands: open, close, up, down")

def main():
    rclpy.init()
    node = GripperController()
    if len(sys.argv) > 1:
        cmd = sys.argv[1].lower()
        time.sleep(0.5)
        node.send_cmd(cmd)
        time.sleep(0.5)
```

```
    else:
        print("Usage: python3 gripper_controller.py [open|close|up|down]")
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

References

[1] Saska, Martin, Tomas Krajnik, and Libor Pfeucil. "Cooperative UAV-UGV autonomous indoor surveillance." International Multi-Conference on Systems, Signals & Devices. IEEE, 2012.

[2] Arbanas, Barbara, et al. "Decentralized planning and control for UAV–UGV cooperative teams." Autonomous Robots 42.8 (2018): 1601-1618.

[3] Guinand, Frédéric, François Guérin, and Etienne Petitprez. "UAV-UGV multi-robot system for warehouse inventory: Scheduling issues." International Conference on Computational Collective Intelligence. Cham: Springer International Publishing, 2021.

[4] Herrera, Brad Steven, et al. "Visual Mapping and Autonomous Navigation Using AprilTags in Omnidirectional Mobile Robots: A Realistic ROS-Gazebo Simulation Framework." Engineering Proceedings 115.1 (2025): 5.

[5] L. Zhang, et al., "2D Lidar-Based SLAM and Path Planning for Indoor Rescue Using Mobile Robots," Journal of Advanced Transportation, vol. 2020, 2020.

[6] Fagundes-Júnior, Leonardo A., et al. "Uav–ugv formation for delivery missions: A practical case study." Drones 9.1 (2025): 48.